

EJEMPLO 2

USO DE VERT.X EN NUESTRO PROYECTO

1. Implementación en el proyecto
 - a. Creación del Verticle principal.

```
© MainVerticle.java x
1 package dawflix.recomendador;
2
3 import io.vertx.core.AbstractVerticle;
4 import io.vertx.core.Promise;
5 import io.vertx.core.json.JsonArray;
6 import io.vertx.core.json.JsonObject;
7 import io.vertx.ext.web.Router;
8 import io.vertx.ext.web.handler.BodyHandler;
9
10 public class MainVerticle extends AbstractVerticle { 1 usage new *
11
12     @Override new *
13     @
14     public void start(Promise<Void> startPromise) {
15         Router router = Router.router(vertx);
16
17         // Habilitar CORS
18         router.route().handler(ctx -> {...});
19
20
21         router.route().handler(BodyHandler.create());
22
23
24         // Ruta de recomendaciones
25         router.get("/recomendar").handler(ctx -> {...});
26
27
28         vertx.createHttpServer().requestHandler(router)
29             .listen(port: 8888) Future<HttpServer>
30             .onSuccess(server -> {
31                 System.out.println("Servidor de recomendaciones activo en http://localhost:8888");
32                 startPromise.complete();
33             })
34             .onFailure(startPromise::fail);
35     }
36 }
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51 }
```

Creamos la clase **MainVerticle**, que actúa como el servidor HTTP donde se define una ruta REST (**/recomendar**) para devolver recomendaciones. Este verticle gestiona peticiones y respuestas de forma asíncrona y fija (para simplificarlo más). Escucha peticiones HTTP en el puerto 8888 y responde con un JSON de películas recomendadas.

b. Creación del Launcher.

```
Launcher.java x
1 package dawflix.recomendador;
2
3 import io.vertx.core.Vertx;
4
5 public class Launcher { new *
6     public static void main(String[] args) { new *
7         Vertx vertx = Vertx.vertx();
8         vertx.deployVerticle(new MainVerticle());
9     }
10 }
```

Launcher es una clase Java que lanza el **MainVerticle**. Sirve como punto de entrada del microservicio.

c. Añadir dependencias y plugin en el pom.xml.

```
<!-- Core Vert.x -->
<dependency>
    <groupId>io.vertx</groupId>
    <artifactId>vertx-core</artifactId>
    <version>4.5.1</version>
</dependency>

<!-- Web para router y servidor HTTP -->
<dependency>
    <groupId>io.vertx</groupId>
    <artifactId>vertx-web</artifactId>
    <version>4.5.1</version>
</dependency>
```

```
<plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <version>3.1.0</version>
    <configuration>
        <mainClass>dawflix.recomendador.Launcher</mainClass>
    </configuration>
</plugin>
```

Incluimos las dependencias necesarias de Vert.x como **vertx-core**, **vertx-web**, y el plugin **exec-maven-plugin**.

d. Crear el script en js.

```
1 document.addEventListener({ type: "DOMContentLoaded", listener: () => { new *
2   console.log("Cargando recomendaciones desde Vert.x");
3
4   const idPelicula :string = new URLSearchParams(window.location.search).get("idPelicula");
5   if (!idPelicula) return;
6
7   fetch({ input: 'http://localhost:8888/recomendar?id=${idPelicula}' }) Promise<Response>
8     .then(res :Response => {
9       if (!res.ok) throw new Error("Error al obtener recomendaciones");
10      return res.json();
11    }) Promise<any>
12    .then(peliculas => {
13      const contenedor :Element = document.querySelector( selectors: "#recomendacionesVertx");
14      if (!contenedor || !Array.isArray(peliculas)) return;
15
16      peliculas.forEach(p => {
17        const card :HTMLElement = document.createElement( tagName: "div");
18        card.className = "m-2 text-center";
19        card.style.width = "150px";
20
21        card.innerHTML = `
22          <a href="paginaPelicula.xhtml?idPelicula=${p.id}" class="text-decoration-none text-dark">
23            
24            <div class="mt-2"><strong>${p.titulo}</strong></div>
25          </a>
26        `;
27
28        contenedor.appendChild(card);
29      });
30    }) Promise<void>
31    .catch(err => {
32      console.error("Error cargando recomendaciones:", err);
33    });
34 });
```

Este script se ejecuta al cargar la vista de una película. Hace una petición **fetch()** al servidor Vert.x para obtener recomendaciones relacionadas.

e. Modificar vista.

```
<div class="card bg-white text-dark shadow-lg rounded p-3 mt-4">
  <div class="card-header bg-primary text-white rounded">
    <h4>Películas relacionadas</h4>
  </div>
  <div class="card-body">
    <div class="d-flex flex-wrap">
      <div id="recomendacionesVertx" class="d-flex flex-wrap">
        <!-- Aquí se insertarán las películas recomendadas por JS -->
      </div>
    </div>
  </div>
</div>
```

```
<script src="#{request.contextPath}/resources/js/recomendaciones.js"></script>
```

Se ha añadido un **div** vacío con ID **recomendacionesVertx** donde se inyectan las recomendaciones, y se ha vinculado el script con **<script>**.

f. Lanzar servidor de recomendaciones.

```
C:\Users\carlo\Desktop\UNIVERSIDAD\3AÑO\2CUATRI\DESARROLLO DE APLICACIONES WEB\PROYECTO>mvn compile exec:java -Dexec.mainClass=dawflix.recomendador.Launcher
[INFO] Scanning for projects...
[INFO]
[INFO] -----< daw:dawflix >-----
[INFO] Building app 1.0
[INFO] from pom.xml
[INFO] -----[ war ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ dawflix ---
[INFO] Copying 2 resources from src/main/resources to target/classes
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ dawflix ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- exec:3.1.0:java (default-cli) @ dawflix ---
Servidor de recomendaciones activo en http://localhost:8888
```

Desde la terminal, ejecutamos el comando **mvn compile exec:java -Dexec.mainClass=...** para iniciar el servidor Vert.x.

2. Resultado de como se ve implementado en la aplicación:



Título: Origen

Director: Christopher Nolan
Fecha de estreno: 2010-07-16
Género: Ciencia_Ficcion
Duración: 148 minutos
Valoración: 4.6
Precio: 5.99
Disponibilidad: true

Modificar película

Sinopsis

Un ladrón de sueños intenta implantar una idea en la mente de un objetivo.

Películas relacionadas



El señor de los anillos



Intertellar



El club de la lucha

3. Si quisiéramos cambiar la funcionalidad de como se muestran las películas recomendadas, solo habría que cerrar el servidor que acabamos de crear y procederíamos a cambiarlo. Esto no afectaría en nada al rendimiento de la aplicación, que podría seguir funcionando sin esta funcionalidad, como se muestra en la siguiente imagen:



Título: Origen

Director: Christopher Nolan
Fecha de estreno: 2010-07-16
Género: Ciencia_Ficción
Duración: 148 minutos
Valoración: 4.6
Precio: 5.99
Disponibilidad: true

Modificar película

Sinopsis

Un ladrón de sueños intenta implantar una idea en la mente de un objetivo.

Películas relacionadas

4. ¿Qué hemos conseguido con esto?

Gracias a esta integración de Vert.x, hemos logrado separar el sistema de recomendaciones del núcleo principal de la aplicación, permitiendo que se ejecute en un servidor independiente. Esto reduce la carga del backend principal y mejora la capacidad de respuesta para el usuario.

Antes, las recomendaciones se generaban directamente desde JSF con datos precargados, lo que limitaba la flexibilidad y hacía que cada cambio o cálculo dependiera del ciclo de vida completo de JSF. Ahora, el contenido recomendado se obtiene de forma asíncrona, sin recargar la página, lo que mejora notablemente la experiencia de usuario.

Esto lo podemos replicar en otras funcionalidades como los comentarios, la gestión de pagos o los alquileres y haríamos la aplicación más modular y escalable.